

ChefGaa Integration Guide

Desi Dhamaka Website & Admin CMS — July 7, 2026

What it does (one sentence)

ChefGaa is the source of truth for menu data (categories, items, prices, availability, images). The CMS pulls that data on a schedule or on demand, stores it in Supabase, and the public website reads from Supabase — it never talks to ChefGaa directly.

Big picture

ChefGaa POS (Legacy API for South Plainfield & Oak Tree, V2 API for Lawrenceville) !' Netlify Functions (chefgaa-sync manual + chefgaa-sync-scheduled every 15 min) !' Orchestrator (lock, queue, health checks) !' Supabase (menu_categories, menu_items, audit tables) !' Public website menu page. Customers order via deep links to ChefGaa ordering sites.

The three locations — two API types

Location	API	Identifier
South Plainfield	Legacy	Outlet 70, order type 106
Oak Tree	Legacy	Outlet 71, order type 108
Lawrenceville	V2	Tenant UUID + Store UUID + platform slug

Legacy API: GET <https://api.chefgaa.com/menu-item> with outlet and partner headers — full menu in one call. V2 API: GET <https://chf2-customer-api.chefgaa.com/> with tenant-id, store-id, and x-platform headers. South Plainfield and Oak Tree are production-ready on Legacy. Lawrenceville uses V2 — credentials must match what ChefGaa expects.

What gets synced

- Category name, slug, display order, active/inactive status
- Item name, description, price, image URL
- Availability (available / unavailable)
- Inferred fields: veg/non-veg, spice level, chef special, popular flag
- External IDs: chefgaa_category_id, chefgaa_outlet_item_id
- Content hash per row so unchanged items are skipped

Not synced to the website: raw ChefGaa JSON, payment secrets, modifier groups (future work).

Step-by-step: what happens when sync runs

1. Trigger — manual (admin button) or scheduled (every 15 minutes via Netlify cron).
2. Auth (manual only) — admin must be logged in; Netlify function verifies Supabase JWT and admin/staff role.
3. Lock — only one sync at a time (chefgaa_sync_lock). Manual sync queues if busy; scheduled sync skips.
4. Health check — pings each location ChefGaa API before syncing.
5. Download — fetches menu from Legacy or V2 client per location config.
6. Normalize — maps ChefGaa format into standard internal catalog (mapper layer).
7. First sync only — archives old CMS menu rows into menu_*_legacy_archive tables.
8. Diff & upsert — creates new items, updates changed ones, deactivates removed items.
9. Audit — writes to chefgaa_sync_runs and chefgaa_sync_run_events.
10. Cache bust — bumps catalog_revision so public site picks up fresh data within ~60 seconds.

Admin UI

Path: <https://desi-dhamaka-admin.netlify.app/admin/integrations/chefgaa>

Feature	What it means
Location cards	Per-store status: connected / failed / never synced
Sync All	Queues sync for all enabled locations
Sync location	Sync one store only
Metrics	Last sync time, duration, category/item counts
Sync history	Full audit log — filter by location or result
Auto-refresh	Dashboard polls every 60 seconds

The admin UI calls POST `/netlify/functions/chefgaa-sync` with the admin auth token. Sync never runs in the browser.

How the public website uses it

- User visits `/south-plainfield/menu`.
- `menuPublic.ts` reads `menu_categories` + `menu_items` from Supabase (filtered by `location_id`).
- In-memory cache (~60s) checks `catalog_revision` — if sync bumped it, cache clears.
- If Supabase is empty, falls back to static seed data.
- Order Now links go to ChefGaa ordering site with deep-link params for category/item names.

The website menu is display + discovery. Actual ordering/checkout happens on ChefGaa.

Database tables

Table	Purpose
<code>chefgaa_location_config</code>	Per-location API version, outlet/tenant IDs, sync settings
<code>chefgaa_sync_runs</code>	Each sync attempt — counts, duration, success/partial/failed
<code>chefgaa_sync_run_events</code>	Detailed log lines per run
<code>chefgaa_sync_lock</code>	Singleton lock — prevents overlapping syncs
<code>chefgaa_sync_queue</code>	Pending manual syncs when busy
<code>chefgaa_sync_notifications</code>	Events: price changes, new items, failures
<code>chefgaa_api_health_checks</code>	API reachability audit
<code>menu_categories</code> / <code>menu_items</code>	Normalized menu + ChefGaa external IDs
<code>menu_*_legacy_archive</code>	Backup of pre-ChefGaa CMS data

Safety rules built in

Rule	Why
Server-side only	ChefGaa API calls use Netlify functions + service role key
Admin auth required	Manual sync needs logged-in admin/staff
<code>manual_override</code> per item	Sync won't overwrite that item's CMS edits
Content hash diffing	Unchanged items skip DB updates
First-sync archive	Old CMS menu isn't lost — moved to archive tables
Location isolation	All menu rows scoped by <code>location_id</code>
Lock + queue	No race conditions from double-clicks or cron overlap

Scheduling & local commands

- Netlify scheduled function: chefgaa-sync-scheduled runs every 15 minutes (* / 15 * * * *).
- Default sync_schedule in DB is 15m for all locations.
- Local: npm run sync:chefgaa or npm run sync:chefgaa:all

Environment variables (production)

Variable	Used for
SUPABASE_SERVICE_ROLE_KEY	Server-side sync writes (bypasses RLS)
VITE_SUPABASE_URL / SUPABASE_URL	Supabase connection
Netlify deploy	Functions must be deployed (not just static build)

No ChefGaa API key needed for public menu reads on Legacy — outlet headers are enough. V2 needs correct tenant/store/platform config from ChefGaa admin.

FAQ — likely dev team questions

Is ChefGaa the CMS or is our CMS the CMS?

ChefGaa owns menu/pricing. Our CMS is a sync dashboard + optional overrides. Marketing copy, homepage, SEO, gallery, etc. stay in our CMS.

What if sync fails?

Last good data stays in Supabase. Run is logged as failed or partial. Admin can retry manually. Scheduled job retries in 15 minutes.

What if an admin edits a menu item in our CMS?

Unless manual_override = true on that item, the next sync will overwrite it with ChefGaa data. ChefGaa is source of truth by design.

What happens when ChefGaa removes an item?

Item is deactivated (soft-delete via chefgaa_removed_at / status inactive), not hard-deleted.

Why two API versions?

Older NJ locations (SP, Oak Tree) use Legacy. Lawrenceville was onboarded on ChefGaa V2. The integration has an adapter per location — same normalized output regardless of API.

Does the public site hit ChefGaa on every page load?

No. It reads Supabase only. ChefGaa is hit only during sync (server-side).

Can we sync one location without affecting others?

Yes — use Sync location on that card. Each location syncs independently.

What about orders / payments?

Not integrated. We deep-link to ChefGaa ordering. No order webhooks or payment flow in this integration.

Where is the code?

Integration: src/integrations/chefgaa/. Admin UI: src/admin/pages/ChefGaaIntegrationPage.tsx. Netlify functions: netlify/functions/chefgaa-sync.mts and chefgaa-sync-scheduled.mts. Migrations: 017, 018, 019.

One-line pitch for stakeholders

"Our website menu auto-updates from ChefGaa every 15 minutes. Staff manage the menu in ChefGaa; our CMS pulls it in and shows it on the site. Customers still order through ChefGaa ordering platform."